

Architecting and Querying a Clinical Database Using Python and SQL

I completed a project to create and query clinical database. Following are the steps in this project.

1. Conceptualize the database.
2. Generating artificial data for the tables using Python.
3. Creating the database from these tables in sqlite.
4. Querying the database using SQL.

Following is the detailed description of the above steps.

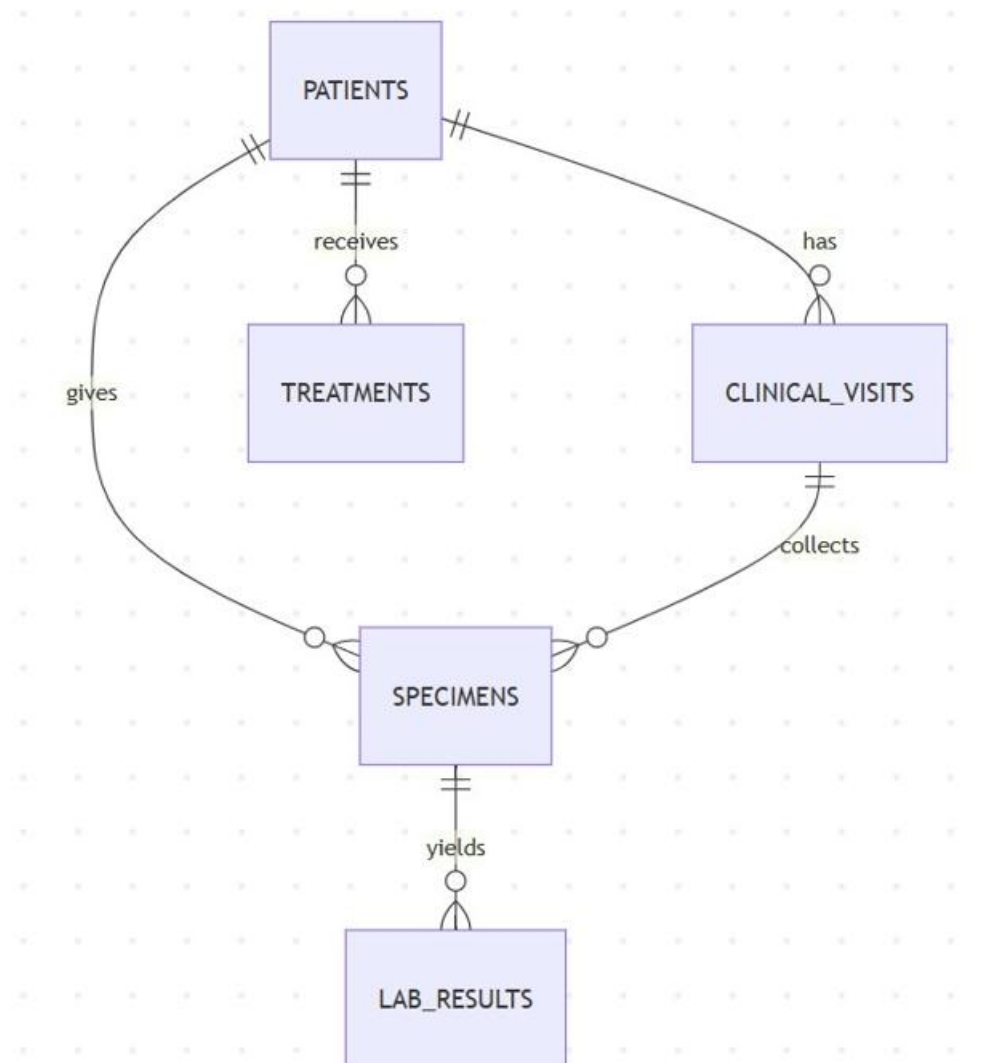
1. Conceptualize the database.

As I planned to design a clinical database, I decided to include 5 tables in it.

The tables are 'Patients', 'Clinical Visits', 'Specimens', 'Lab Results', 'Treatment'.

I included these tables because they depict the whole picture in a clinical setting right from the incoming patients till getting the treatment.

Following is the Entity Relationship Diagram showing connections between these tables.



After deciding about the tables and the relationship between them, I thought about which attributes I can add to each table. Since I had experience with clinical data processing, I was able to come up with meaningful attributes which mimic real world scenarios.

Following are the 'create table' commands showing tables with their attributes. Using these SQL commands, I created empty tables in sqlite.

```
CREATE TABLE Patients (  
    patient_id VARCHAR(50) PRIMARY KEY,  
    age INT NOT NULL,  
    gender VARCHAR(20) NOT NULL,  
    height DECIMAL(5,2),  
    weight DECIMAL(5,2),
```

```
    smoking_status VARCHAR(50)  
    CHECK (smoking_status IN (  
        'frequent_smoker',  
        'occasional_smoker',  
        'non_smoker',  
        'past_smoker'  
    )),
```

```
    alcohol_intake_status VARCHAR(50)  
    CHECK (alcohol_intake_status IN (  
        'frequent_alcohol_intake',  
        'occasional_alcohol_intake',  
        'no_alcohol_intake',  
        'past_alcohol_intake'
```

)),

biological_cohort VARCHAR(50) NOT NULL

CHECK (biological_cohort IN (

'oncology cohort',

'maternal_fetal cohort'

))

);

CREATE TABLE Clinical_Visits (

visit_id VARCHAR(50) PRIMARY KEY,

patient_id VARCHAR(50) NOT NULL,

visit_date DATE NOT NULL,

clinical_information VARCHAR(255) NOT NULL,

-- Establish the relationship to the Patients table

FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)

);

CREATE TABLE Specimens (

specimen_id VARCHAR(50) PRIMARY KEY,

visit_id VARCHAR(50) NOT NULL,

specimen_type VARCHAR(50) NOT NULL

CHECK (specimen_type IN (

'biofluid',

```
'solid tumor',  
'solid normal',  
'cellular'  
)),
```

specimen_category VARCHAR(50), -- Allows NULL, kept open for evolving vocabularies

body_location VARCHAR(100), -- Allows NULL (e.g., 'Left Lung', 'Right Colon')

procurement_procedure VARCHAR(100), -- Allows NULL, kept open for varying clinical workflows

specimen_amount DECIMAL(10,2), -- Allows NULL, supports precise amounts like 4.50

specimen_unit VARCHAR(10) -- Allows NULL

```
CHECK (specimen_unit IN ('ml', 'mg')),
```

qc_status VARCHAR(20) -- Allows NULL

```
CHECK (qc_status IN (
```

```
'pass',
```

```
'fail',
```

```
'pending',
```

```
'marginal',
```

```
'rejected'
```

```
)),
```

-- Establish the relationship back to the timeline

```
FOREIGN KEY (visit_id) REFERENCES Clinical_Visits(visit_id)
);

CREATE TABLE Lab_Results (
    result_id VARCHAR(50) PRIMARY KEY,
    specimen_id VARCHAR(50) NOT NULL,

    test_category VARCHAR(100) NOT NULL,
    -- e.g., 'NIPT Panel', 'Immunohistochemistry (IHC)', 'Liquid Biopsy Marker'

    analyte_name VARCHAR(100) NOT NULL,
    -- e.g., 'Fetal Fraction', 'Trisomy 21', 'HER2 Expression', 'CA-125'

    result_value DECIMAL(10,4),
    -- Allows NULL for qualitative results (e.g., leaving blank for a 'Trisomy 21' test)

    result_unit VARCHAR(50),
    -- Allows NULL (e.g., '%', 'U/mL', 'Score (0-3+)')

    clinical_interpretation VARCHAR(255),
    -- Allows NULL (e.g., 'Low Risk', 'High Risk', 'Elevated', 'Positive')

    test_status VARCHAR(50) NOT NULL

    CHECK (test_status IN (
        'completed',
        'cancelled',
        'in_progress',
```

'inconclusive'

)),

-- Establish the relationship back to the physical sample

FOREIGN KEY (specimen_id) REFERENCES Specimens(specimen_id)

);

CREATE TABLE Treatments (

treatment_id VARCHAR(50) PRIMARY KEY,

patient_id VARCHAR(50) NOT NULL,

visit_id VARCHAR(50), -- Allows NULL, but useful to link when the treatment was prescribed

treatment_category VARCHAR(100) NOT NULL,

-- e.g., 'Targeted Therapy', 'Immunotherapy', 'Chemotherapy', 'Prenatal Care', 'Surgical Intervention'

treatment_name VARCHAR(150) NOT NULL,

-- e.g., 'Trastuzumab', 'Pembrolizumab', 'Routine Prenatal Vitamins'

start_date DATE NOT NULL,

end_date DATE,

-- Allows NULL. A null end_date indicates the patient is currently actively receiving the treatment

treatment_outcome VARCHAR(100)

-- Allows NULL. e.g., 'Complete Response', 'Disease Progression', 'Routine Care Completed'

CHECK (treatment_outcome IN (

'Complete Response',

'Partial Response',

'Stable Disease',

'Disease Progression',

'Adverse Event - Discontinued',

'Routine Care Completed'

)),

-- Establish relationships back to the patient and the timeline

FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),

FOREIGN KEY (visit_id) REFERENCES Clinical_Visits(visit_id)

);

One can see how I have used Primary Keys, Foreign Keys and some constraints to keep integrity of the database and to keep it in good shape.

2. Generating artificial data for the tables using Python

I decided to use libraries in Python to generate artificial data. I got the help of Google Gemini to write the code for this task. Before using it, I thoroughly checked the code, tweaked it wherever necessary and I added many helpful comments to understand and navigate through the program efficiently.

Following is the code to create 'Patients' table.

```
# Import necessary libraries in Python

import pandas as pd
import numpy as np
from faker import Faker
from datetime import timedelta

## Faker is an incredible library for generating realistic mock data (like unique IDs and
dates).

# timedelta allows us to do "date math" (e.g., adding 30 days to a specific date), which is
crucial for building a timeline.

# Set a random seed so the results are reproducible if you run it multiple times
np.random.seed(42)

# Number of records to generate
NUM_PATIENTS = 200

# Initialize an empty list to store our rows
data = [] #We initialize an empty Python list. As we generate each patient inside the
upcoming loop, we will append their record to this list.

# Generate the cohort distribution (e.g., 70% oncology, 30% maternal-fetal)
cohorts = np.random.choice(
    ['oncology cohort', 'maternal_fetal cohort'],
```

```

size=NUM_PATIENTS,

p=[0.7, 0.3] #The np.random.choice function picks 200 values from the provided list, but
the p=[0.7, 0.3] argument applies "weights."

    #This ensures that roughly 70% of your dataset will be assigned to the oncology
cohort and 30% to the maternal-fetal cohort,

    #making it statistically realistic.
)

for i, cohort in enumerate(cohorts):

    # 1. patient_id (NOT NULL)

    patient_id = f"PT-{i+1:05d}" # Generates PT-00001, PT-00002, etc.

    # 2 & 3. age and gender (NOT NULL) - tailored by cohort
    if cohort == 'maternal_fetal cohort':

        age = int(np.random.randint(18, 45))

        gender = 'Female'

    else:

        age = int(np.random.randint(40, 85))

        gender = np.random.choice(['Male', 'Female'])

#The Loop (for i, cohort in enumerate): This iterates 200 times. i is the current count (0, 1,
2...), and cohort is the specific cohort assigned to that

#patient in the previous step.

#Formatted IDs: f"PT-{i+1:05d}" creates the ID. i+1 ensures we start at 1 instead of 0. The
:05d tells Python to pad the number with leading zeros

#up to 5 digits (e.g., PT-00001).

```

#Biological Business Logic: The if/else block acts as a clinical guardrail. If a patient is in the maternal-fetal cohort,

#they must be assigned as 'Female' and their age is restricted to a realistic reproductive window (18 to 44).

#Oncology patients are given an older age range (40 to 84) and can be male or female.

4 & 5. height (cm) and weight (kg) (NULLABLE)

Generate realistic base numbers depending on gender

if gender == 'Male':

height = round(np.random.normal(175.0, 7.0), 2)

weight = round(np.random.normal(82.0, 15.0), 2)

else:

height = round(np.random.normal(162.0, 7.0), 2)

weight = round(np.random.normal(68.0, 15.0), 2)

#Instead of picking a random number between two endpoints, np.random.normal

#generates a number based on a normal distribution (also known as a bell curve). It takes two main arguments:

#The Mean (Average): The first number (175.0). This is the peak of the bell curve. Most patients will be generated very close to this number.

#The Standard Deviation: The second number (7.0). This dictates how wide the bell curve is.

#A standard deviation of 7 means that about 68% of the generated male patients will fall within 7 units of the mean

#(between 168.0 and 182.0), and 95% will fall within 14 units. It allows for natural variation while preventing extreme outliers.

#3. The Specific Clinical Values (cm and kg)

#The numbers chosen here represent real-world clinical averages (roughly based on global/US averages):

#For Men: Average height of 175.0 cm (about 5'9") and average weight of 82.0 kg (about 180 lbs).

#For Women: Average height of 162.0 cm (about 5'4") and average weight of 68.0 kg (about 150 lbs).

#The spread (15 kg standard deviation for weight) accounts for the fact that human weight naturally varies much more drastically than

#human height (7 cm standard deviation).

```
# Introduce ~15% missing data (NULLs) for height and weight
```

```
if np.random.random() < 0.15:
```

```
    height = np.nan
```

```
if np.random.random() < 0.15:
```

```
    weight = np.nan
```

#After generating the height and weight using the statistical bell curves, the script intentionally breaks some of the data.

#np.random.random() generates a single decimal between 0.0 and 1.0.

#Checking if it is < 0.15 creates a 15% probability condition.

#If the condition is met, the height or weight is replaced with np.nan (Not a Number), which is how Pandas represents missing data.

```
# 6. smoking_status (NULLABLE)
```

```

smoking_options = ['frequent_smoker', 'occasional_smoker', 'non_smoker',
'past_smoker']

# Create a realistic distribution skewed towards non/past smokers
smoking = np.random.choice(smoking_options, p=[0.10, 0.10, 0.55, 0.25])
if np.random.random() < 0.20: # 20% chance the patient skipped this question
    smoking = np.nan

#Much like the cohort distribution, we use np.random.choice with the p= argument to
create realistic lifestyle populations.

#Instead of assuming 25% of the population falls into each of the four smoking categories,
we weight it so 55% are non-smokers

#and 25% are past smokers, which closer aligns with modern hospital demographics.

#We then apply a slightly higher missing data rate (20%) to this column, simulating the fact
that patients frequently skip

#lifestyle questions on intake forms.


# 7. alcohol_intake_status (NULLABLE)

alcohol_options = ['frequent_alcohol_intake', 'occasional_alcohol_intake',
'no_alcohol_intake', 'past_alcohol_intake']

alcohol = np.random.choice(alcohol_options, p=[0.05, 0.30, 0.45, 0.20])
if np.random.random() < 0.20: # 20% chance the patient skipped this question
    alcohol = np.nan

#Similar process for alcohol_intake as that of smoking_status


# Append the row to our data list
data.append([
    patient_id, age, gender, height, weight,

```

```
        smoking, alcohol, cohort
    ])

# Create the DataFrame
columns = [
    'patient_id', 'age', 'gender', 'height', 'weight',
    'smoking_status', 'alcohol_intake_status', 'biological_cohort'
]
df = pd.DataFrame(data, columns=columns)

# Preview the first few rows
print(df.head(10))

# Export to CSV
# Using na_rep="" ensures that np.nan values are written as empty strings,
# which SQL will perfectly interpret as NULLs during import.
df.to_csv('patients_data.csv', index=False, na_rep="")
```

Sample 'Patients' table.

	A	B	C	D	E	F	G	H
1	patient_id	age	gender	height	weight	smoking_status	alcohol_intake_status	biological_cohort
2	PT-00001	63	Female				occasional alcohol intake	oncology cohort
3	PT-00002	39	Female	163.05			no alcohol intake	maternal fetal cohort
4	PT-00003	20	Female		96.45	non smoker	past alcohol intake	maternal fetal cohort
5	PT-00004	47	Male	161.45	69.29	past smoker	no alcohol intake	oncology cohort
6	PT-00005	56	Female		78.71	non smoker	past alcohol intake	oncology cohort
7	PT-00006	60	Female	158.87	80.85	past smoker		oncology cohort
8	PT-00007	71	Male	182.4	82.77	non smoker	occasional alcohol intake	oncology cohort
9	PT-00008	20	Female				occasional alcohol intake	maternal fetal cohort
10	PT-00009	53	Female	151.08	98.01	occasional_smoker	past alcohol intake	oncology cohort

Following is the code to create 'Clinical Visits' table.

```
# Initialize Faker and random seeds for reproducibility

fake = Faker()

Faker.seed(42)

np.random.seed(42)


#Random number generators aren't truly random; they follow an algorithm. By setting a
"seed," you lock that algorithm.

#This means every time you run this script, it will generate the exact same 200 patients.

#This is great for debugging or sharing your code with an interviewer.


# 1. Load the existing patients data to ensure perfect alignment
try:

    df_patients = pd.read_csv('patients_data.csv')

except FileNotFoundError:

    print("Error: 'patients_data.csv' not found. Please run the patients script first.")

    exit()
```

```

visits_data = []

# 2. Iterate through each patient to build a longitudinal timeline
for index, row in df_patients.iterrows():

    patient_id = row['patient_id']

    cohort = row['biological_cohort']

    # Generate a random baseline date within the last 3 years
    baseline_date = fake.date_between(start_date='-3y', end_date='-1y')

# iterrows(): This loops through every single row in the patients table, grabbing their
patient_id and their biological_cohort.

#The Baseline Date: fake.date_between generates a random starting date for each patient's
medical journey, somewhere between 1 and 3 years ago.

#The First Record: Every single patient, regardless of their cohort, gets this initial 'Baseline
Initial Consult' appended to the master data list.

#This represents the day they first walked into the clinic as follows.

# Every patient gets at least a baseline visit
visit_id_base = f"V-{fake.unique.random_number(digits=6, fix_len=True)}"
visits_data.append([visit_id_base, patient_id, baseline_date, 'Baseline Initial Consult'])

# 3. Generate follow-up visits based on their specific cohort
if cohort == 'oncology cohort':

    # Oncology patients might have 1 to 4 follow-up visits
    num_followups = np.random.randint(1, 5)

```



```
current_date = baseline_date
```

```
oncology_events = ['Mid-Treatment Evaluation', 'Post-Treatment Follow-up', 'Routine Monitoring', 'Suspected Relapse']
```

```
for i in range(num_followups):
```

```
    visit_id_fu = f"V-{fake.unique.random_number(digits=6, fix_len=True)}"
```

```
    # Advance the date by 30 to 90 days for each subsequent visit
```

```
    current_date += timedelta(days=np.random.randint(30, 90))
```

```
    # Pick a logical progression of clinical events
```

```
    clinical_info = oncology_events[min(i, len(oncology_events)-1)]
```

```
    visits_data.append([visit_id_fu, patient_id, current_date, clinical_info])
```

#Determining Visit Count: np.random.randint(1, 5) decides that an oncology patient will have between 1 and 4 follow-up visits.

#The Time Jump (timedelta): Inside the follow-up loop, we take the current_date (which starts as the baseline date) and add anywhere

#from 30 to 90 days to it.

#This ensures that Visit 2 happens after Visit 1, Visit 3 happens after Visit 2, and so on. Without this, your dates would be a jumbled mess.

#Sequential Events: The oncology_events list acts as a storyline. The min(i, len...) math ensures that the script assigns these events in order.

#The first follow-up is a 'Mid-Treatment Evaluation', the next is 'Post-Treatment', etc. If they have 4 visits,

#the final one becomes a 'Suspected Relapse'.

```

elif cohort == 'maternal_fetal cohort':

    # Maternal/Fetal patients generally have a structured schedule

    # Generating 1 to 3 follow-ups representing trimesters/screenings
    num_followups = np.random.randint(1, 4)

    current_date = baseline_date

    maternal_events = ['First Trimester Anatomy Scan', 'Second Trimester NIPT Draw', 'Third
Trimester Checkup']

    for i in range(num_followups):

        visit_id_fu = f"V-{fake.unique.random_number(digits=6, fix_len=True)}"

        # Advance the date by 20 to 45 days
        current_date += timedelta(days=np.random.randint(20, 45))

        clinical_info = maternal_events[min(i, len(maternal_events)-1)]

        visits_data.append([visit_id_fu, patient_id, current_date, clinical_info])

#Cohort Specifics: This follows the exact same logic structure as the oncology block, but
the parameters change to reflect the biology.

#Maternal-fetal patients get fewer maximum visits (1 to 3 follow-ups).

#The time jump is shorter (20 to 45 days) because prenatal checkups happen more
frequently than long-term cancer monitoring.

#The events are entirely swapped out for trimester-specific milestones.

# 4. Create the DataFrame
columns = ['visit_id', 'patient_id', 'visit_date', 'clinical_information']

```

```
df_visits = pd.DataFrame(visits_data, columns=columns)

# Sort by patient_id and visit_date so the timeline is clean
df_visits = df_visits.sort_values(by=['patient_id', 'visit_date'])

#Sorting (sort_values): This is a critical finishing touch for database readability. Even
though we generated the dates

#chronologically per patient, if you just dumped the data into a CSV, it might look messy.

#By sorting first by patient_id and then by visit_date, the final output will group all of Patient
1's visits together in perfect chronological order,

#then move on to Patient 2.


# Preview the first 10 rows
print(df_visits.head(10))


# 5. Export to CSV
df_visits.to_csv('clinical_visits_data.csv', index=False)

print(f"\nSuccessfully generated {len(df_visits)} clinical visits and saved to
'clinical_visits_data.csv'")
```

Sample 'Clinical Visits' table.

	A	B	C	D
1	visit_id	patient_id	visit_date	clinical_information
2	V-216739	PT-00001	9/3/2024	Baseline Initial Consult
3	V-126225	PT-00001	11/23/2024	Mid-Treatment Evaluation
4	V-877572	PT-00001	1/20/2025	Post-Treatment Follow-up
5	V-388389	PT-00001	3/5/2025	Routine Monitoring
6	V-334053	PT-00002	11/2/2023	Baseline Initial Consult
7	V-246316	PT-00002	11/29/2023	First Trimester Anatomy Scan
8	V-872246	PT-00002	1/8/2024	Second Trimester NIPT Draw
9	V-207473	PT-00002	2/3/2024	Third Trimester Checkup
10	V-876646	PT-00003	10/2/2024	Baseline Initial Consult

Following is the code to create 'Specimens' table.

```
# Initialize Faker and random seed for reproducible data

fake = Faker()

Faker.seed(42)

np.random.seed(42)


# 1. Load the existing datasets to maintain relational integrity
try:

    df_patients = pd.read_csv('patients_data.csv')

    df_visits = pd.read_csv('clinical_visits_data.csv')

except FileNotFoundError:

    print("Error: Please ensure 'patients_data.csv' and 'clinical_visits_data.csv' are in the
    same directory.")

    exit()
```

```
# Merge visits with patients so we know the cohort for each visit

# The clinical_visits table doesn't contain the biological_cohort column (because that
belongs to the patient).

# To write rules based on the cohort, we perform a SQL-style LEFT JOIN using Pandas.

# Now, for every single visit in the timeline, the script knows exactly which cohort that
patient belongs to.


df_merged = df_visits.merge(df_patients[['patient_id', 'biological_cohort']], on='patient_id',
how='left')


specimens_data = []


# List of plausible solid tumor locations

body_locations = ['Right Lung', 'Left Lung', 'Descending Colon', 'Right Breast', 'Left Breast',
'Liver', 'Thyroid']


# 2. Iterate through each visit to generate the corresponding physical specimens
for index, row in df_merged.iterrows():

    visit_id = row['visit_id']

    cohort = row['biological_cohort']

    clinical_info = row['clinical_information']


    specimen_id = f"SP-{fake.unique.random_number(digits=7, fix_len=True)}"


#Iterating the Timeline:

#We loop through every single visit. Every visit triggers the collection of at least one
specimen.
```

#Blank Canvas: We set the default values for the specific columns to np.nan (empty).

#The following if/elif blocks will fill these in based on the medical context.

```
# Initialize defaults
```

```
s_type = "
```

```
s_cat = "
```

```
location = np.nan
```

```
procedure = np.nan
```

```
amount = np.nan
```

```
unit = np.nan
```

```
# =====
```

```
# ONCOLOGY COHORT LOGIC
```

```
# =====
```

```
if cohort == 'oncology cohort':
```

```
    if 'Baseline' in clinical_info:
```

```
        # Baseline visits usually yield a primary tissue biopsy (e.g., for RNA-seq or CGP)
```

```
        s_type = 'solid tumor'
```

```
        s_cat = np.random.choice(['ffpe', 'fresh frozen', 'tissue'], p=[0.6, 0.2, 0.2])
```

```
        procedure = np.random.choice(['Core Biopsy', 'Surgical Resection'])
```

```
        location = np.random.choice(body_locations)
```

```
        amount = round(np.random.uniform(5.0, 50.0), 2)
```

```
        unit = 'mg'
```

#Baseline Detection: If the visit is a "Baseline Initial Consult," the script knows oncology patients need a primary tumor biopsy.

#Specifics: It assigns it as a solid tumor, picks a random category (weighted heavily toward the industry-standard ffpe),

#assigns a realistic surgical procedure, picks a random organ from our body_locations list, and assigns a weight in milligrams (mg).

20% chance they also took an adjacent 'solid normal' tissue sample for germline comparison

```
if np.random.random() < 0.20:
```

```
    norm_spec_id = f"SP-{fake.unique.random_number(digits=7, fix_len=True)}"
```

```
    norm_qc = np.random.choice(['pass', 'marginal', 'pending'], p=[0.8, 0.1, 0.1])
```

```
    specimens_data.append([
```

```
        norm_spec_id, visit_id, 'solid normal', 'tissue', location, procedure,
```

```
        round(np.random.uniform(5.0, 25.0), 2), 'mg', norm_qc
```

```
    ])
```

#The "Solid Normal" Flex: While still inside the baseline block, there is a 20% chance the script generates a second specimen.

#This simulates the surgeon taking a piece of adjacent healthy tissue to compare against the tumor (a standard practice in genomics).

#It appends this second sample immediately, linked to the exact same visit_id.

```
elif 'Relapse' in clinical_info:
```

```
    # A relapse might trigger a localized biofluid extraction (like lung fluid)
```

```
    s_type = 'biofluid'
```

```
    s_cat = 'pleural fluid'
```

```
    procedure = 'Thoracentesis'
```

```

location = 'Left Pleural Cavity' # Here, a biofluid DOES get a location
amount = round(np.random.uniform(15.0, 100.0), 2)
unit = 'ml'

else:

    # Routine monitoring usually relies on liquid biopsy (e.g., tracking MRD via plasma)
    s_type = 'biofluid'
    s_cat = 'plasma'
    procedure = 'Venipuncture'
    location = np.nan # Systemic fluid, no specific location needed
    amount = round(np.random.uniform(4.0, 10.0), 2)
    unit = 'ml'

#Relapse vs. Routine: If the visit isn't a baseline, it checks if it's a "Suspected Relapse"
#(generating a localized fluid with a specific body location) or a routine follow-up
(generating standard, systemic blood plasma with no body location).

# =====
# MATERNAL-FETAL COHORT LOGIC
# =====

elif cohort == 'maternal_fetal cohort':

    if 'Scan' in clinical_info or 'Checkup' in clinical_info:

        # Standard checkups might just take a routine urine sample
        s_type = 'biofluid'
        s_cat = 'urine'
        procedure = 'Midstream Catch'
        location = np.nan

```



```
amount = round(np.random.uniform(20.0, 60.0), 2)
unit = 'ml'
else:
    # NIPT or specific genetic draws require maternal blood/plasma
    s_type = 'biofluid'
    s_cat = 'whole blood'
    procedure = 'Venipuncture'
    location = np.nan
    amount = round(np.random.uniform(8.0, 15.0), 2)
    unit = 'ml'
```

#This is a simpler parallel to the oncology block. Routine checkups generate standard urine samples. High-risk diagnostic draws (like NIPT)

#generate whole blood drawn via Venipuncture.

#Notice how location is left as np.nan for all of these, as they are systemic fluids.

```
# 3. Generate Realistic QC Status (mostly pass, some marginal/fail/pending)
qc_status = np.random.choice(['pass', 'fail', 'marginal', 'pending', 'rejected'], p=[0.75, 0.05, 0.10, 0.08, 0.02])
```

```
# 4. Introduce a few realistic NULLs for operational fields (Amount, Unit, QC)
```

```
if np.random.random() < 0.05: amount = np.nan
```

```
if np.random.random() < 0.05: unit = np.nan
```

```
if np.random.random() < 0.05: qc_status = np.nan
```

```
specimens_data.append([
    specimen_id, visit_id, s_type, s_cat, location,
    procedure, amount, unit, qc_status
```

```
])
```

```
#Quality Control: In the real world, labs mess up. Tubes break, samples hemolyze, or forms get lost.
```

```
#The script assigns a weighted qc_status to every sample, meaning about 25% of your dataset will have operational issues (failed, pending, marginal).
```

```
#Missing Data: Finally, it randomly deletes the amount, unit, or qc_status 5% of the time to simulate data entry errors by laboratory technicians.
```

```
# 5. Create DataFrame and Export
```

```
columns = [
```

```
    'specimen_id', 'visit_id', 'specimen_type', 'specimen_category',
```

```
    'body_location', 'procurement_procedure', 'specimen_amount',
```

```
    'specimen_unit', 'qc_status'
```

```
]
```

```
df_specimens = pd.DataFrame(specimens_data, columns=columns)
```

```
# Preview the data to verify the logic
```

```
print(df_specimens.head(15))
```

```
# Export using na_rep="" so Pandas writes np.nan as empty strings for perfect SQL NULL import
```

```
df_specimens.to_csv('specimens_data.csv', index=False, na_rep="")
```

```
print(f"\nSuccessfully generated {len(df_specimens)} specimen records and saved to 'specimens_data.csv'")
```

Sample 'Specimens' table.

	A	B	C	D	E	F	G	H	I
1	specimen_id	visit_id	specimen_type	specimen_category	body_location	procurement_procedure	specimen_amount	specimen_unit	qc_status
2	SP-2867825	V-216739	solid tumor	ffpe	Thyroid	Core Biopsy	37.94	mg	pass
3	SP-1419610	V-126225	biofluid	plasma		Venipuncture		ml	pass
4	SP-5614226	V-877572	biofluid	plasma		Venipuncture	5.27	ml	pass
5	SP-5108603	V-388389	biofluid	plasma		Venipuncture	6.59	ml	pass
6	SP-4744854	V-334053	biofluid	whole blood		Venipuncture	10.56	ml	pass
7	SP-3341057	V-246316	biofluid	urine		Midstream Catch	43.7	ml	pass
8	SP-2719583	V-872246	biofluid	whole blood		Venipuncture	14.64	ml	pending
9	SP-2458591	V-207473	biofluid	urine		Midstream Catch	47.37	ml	
10	SP-8078673	V-876646	biofluid	whole blood		Venipuncture	14.37	ml	pass

Following is the code to create 'Lab_Results' table.

```
# Initialize Faker and random seed for reproducibility

fake = Faker()

Faker.seed(42)

np.random.seed(42)

# 1. Load the upstream data to maintain complete relational integrity
try:

    df_patients = pd.read_csv('patients_data.csv')

    df_visits = pd.read_csv('clinical_visits_data.csv')

    df_specimens = pd.read_csv('specimens_data.csv')

except FileNotFoundError:

    print("Error: Missing one of the CSV files. Please ensure patients, visits, and specimens
    CSVs are in the directory.")

    exit()

# Merge dataframes to get the cohort mapping for each specimen

# Specimens -> Visits -> Patients
```

```

df_merged = df_specimens.merge(df_visits[['visit_id', 'patient_id']], on='visit_id', how='left')

df_master = df_merged.merge(df_patients[['patient_id', 'biological_cohort']],
on='patient_id', how='left')

lab_results_data = []

# 2. Iterate through each physical specimen to generate corresponding lab tests
for index, row in df_master.iterrows():

    specimen_id = row['specimen_id']
    s_type = row['specimen_type']
    s_cat = row['specimen_category']
    cohort = row['biological_cohort']
    qc_status = row['qc_status']

    # If the specimen failed QC or was rejected at the door, the test is cancelled
    if qc_status in ['fail', 'rejected']:
        result_id = f"RES-{fake.unique.random_number(digits=7, fix_len=True)}"
        lab_results_data.append([
            result_id, specimen_id, 'Lab Triage', 'Sample Viability',
            np.nan, np.nan, 'Quantity Not Sufficient (QNS) or Rejected', 'cancelled'
        ])
        continue # Skip downstream assays for this ruined sample

    # Helper function to assign status (mostly completed, rarely in_progress/inconclusive)
    def get_status():
        return np.random.choice(['completed', 'in_progress', 'inconclusive'], p=[0.90, 0.05,
0.05])

```

```

# =====
# ONCOLOGY COHORT LOGIC
# =====

if cohort == 'oncology cohort':

    # Solid Tumors get Immunohistochemistry (IHC)

    if s_type in ['solid tumor', 'solid normal'] or s_cat in ['ffpe', 'tissue']:

        # Test 1: HER2 Expression

        status_1 = get_status()

        val_1 = float(np.random.choice([0.0, 1.0, 2.0, 3.0])) if status_1 == 'completed' else
np.nan

        interp_1 = np.nan

        if status_1 == 'completed':

            interp_1 = 'Positive' if val_1 == 3.0 else ('Equivocal' if val_1 == 2.0 else 'Negative')

        lab_results_data.append([

            f"RES-{fake.unique.random_number(digits=7, fix_len=True)}", specimen_id,

            'Immunohistochemistry (IHC)', 'HER2 Expression', val_1, 'Score (0-3+)', interp_1,
status_1

        ])

    # Test 2: PD-L1 TPS (percentage)

    status_2 = get_status()

    val_2 = round(np.random.uniform(0.0, 95.0), 2) if status_2 == 'completed' else np.nan

    interp_2 = np.nan

    if status_2 == 'completed':

```

```
interp_2 = 'High Expression' if val_2 >= 50.0 else ('Low Expression' if val_2 >= 1.0
else 'Negative')
```

```
lab_results_data.append([
    f"RES-{fake.unique.random_number(digits=7, fix_len=True)}", specimen_id,
    'Immunohistochemistry (IHC)', 'PD-L1 TPS', val_2, '%', interp_2, status_2
])
```

#HER2: This is scored on a discrete scale (0, 1, 2, 3). The script assigns the number to the result_value column,

#and automatically translates a 3.0 into a "Positive" in the clinical_interpretation column.

#PD-L1: This is a true percentage. The script uses np.random.uniform to generate a random decimal between 0% and 95%.

#It then uses an if/else statement to categorize that number. If the random number is over 50.0, it logs "High Expression."

```
# Biofluids get Liquid Biopsy Markers
```

```
elif s_type == 'biofluid' or s_cat in ['plasma', 'serum', 'pleural fluid']:
```

```
# Test 1: CA-125 (Quantitative)
```

```
status_1 = get_status()
```

```
val_1 = round(np.random.uniform(10.0, 150.0), 2) if status_1 == 'completed' else
np.nan
```

```
interp_1 = 'Elevated' if val_1 > 35.0 else 'Normal' if status_1 == 'completed' else
np.nan
```

```
lab_results_data.append([
    f"RES-{fake.unique.random_number(digits=7, fix_len=True)}", specimen_id,
    'Liquid Biopsy Marker', 'CA-125', val_1, 'U/mL', interp_1, status_1
])
```

```

    ])

    # Test 2: ctDNA Detected (Qualitative - no numbers)

    status_2 = get_status()

    interp_2 = np.random.choice(['Positive (Disease Detected)', 'Negative (No Disease
Detected)'], p=[0.3, 0.7]) if status_2 == 'completed' else np.nan

    lab_results_data.append([

        f"RES-{fake.unique.random_number(digits=7, fix_len=True)}", specimen_id,

        'Liquid Biopsy Marker', 'ctDNA Detected', np.nan, np.nan, interp_2, status_2

    ])

# =====
# MATERNAL-FETAL COHORT LOGIC
# =====

elif cohort == 'maternal_fetal cohort':

    # Blood products get NIPT Panels

    if s_cat in ['whole blood', 'plasma']:

        # Test 1: Fetal Fraction

        status_1 = get_status()

        val_1 = round(np.random.normal(10.0, 3.0), 2) if status_1 == 'completed' else np.nan

        interp_1 = 'Sufficient for Analysis' if val_1 >= 4.0 else 'Quantity Not Sufficient' if
status_1 == 'completed' else np.nan

        lab_results_data.append([

            f"RES-{fake.unique.random_number(digits=7, fix_len=True)}", specimen_id,

            'NIPT Panel', 'Fetal Fraction', val_1, '%', interp_1, status_1

```

```

    ])

    # If Fetal Fraction is sufficient, run the Trisomy risks (Qualitative)
    if status_1 == 'completed' and val_1 >= 4.0:

        # Trisomy 21

        interp_2 = np.random.choice(['Low Risk', 'High Risk'], p=[0.95, 0.05])

        lab_results_data.append([

            f"RES-{fake.unique.random_number(digits=7, fix_len=True)}", specimen_id,

            'NIPT Panel', 'Trisomy 21', np.nan, np.nan, interp_2, 'completed'

        ])

        # Trisomy 18

        interp_3 = np.random.choice(['Low Risk', 'High Risk'], p=[0.98, 0.02])

        lab_results_data.append([

            f"RES-{fake.unique.random_number(digits=7, fix_len=True)}", specimen_id,

            'NIPT Panel', 'Trisomy 18', np.nan, np.nan, interp_3, 'completed'

        ])

    ])

    # Urine samples get a standard Urinalysis
    elif s_cat == 'urine':

        status_1 = get_status()

        interp_1 = np.random.choice(['Negative', 'Trace', '1+'], p=[0.7, 0.2, 0.1]) if status_1 ==
'completed' else np.nan

        lab_results_data.append([

            f"RES-{fake.unique.random_number(digits=7, fix_len=True)}", specimen_id,

            'Urinalysis', 'Protein', np.nan, np.nan, interp_1, status_1

        ])

```


#For a Non-Invasive Prenatal Test (NIPT), the lab must first check if there is enough of the baby's DNA in the mother's blood.

#This is the Fetal Fraction. The script generates this number using a bell curve averaging 10%.

#The Gate: Look at the if ... and val_1 >= 4.0: statement. The script only generates Trisomy 21 and 18 results if the Fetal Fraction is 4% or higher.

#If it's below 4%, the script stops testing that sample, perfectly mirroring clinical safety guidelines.

#Notice how the Trisomy append block assigns np.nan to the result_value and result_unit. Risk assessments don't get numerical values in this schema;

#they rely entirely on the clinical_interpretation column.

3. Create DataFrame and Export

```
columns = [  
    'result_id', 'specimen_id', 'test_category', 'analyte_name',  
    'result_value', 'result_unit', 'clinical_interpretation', 'test_status'  
]
```

```
df_results = pd.DataFrame(lab_results_data, columns=columns)
```

```
print(df_results.head(15))
```

Export using na_rep="" ensures SQL interprets empty strings as NULLs

```
df_results.to_csv('lab_results_data.csv', index=False, na_rep="")
```

#The na_rep="" ensures that qualitative tests (like Trisomy) leave the numerical columns perfectly blank for SQL NULL importation.

```
print(f"\nSuccessfully generated {len(df_results)} lab result records and saved to  
'lab_results_data.csv'")
```

Sample 'Lab_Results' table.

	A	B	C	D	E	F	G	H
1	result_id	specimen_id	test_category	analyte_name	result_value	result_unit	clinical_interpretation	test_status
2	RES-2867825	SP-2867825	Immunohistochemistry (IHC)	HER2 Expression	0	Score (0-3+)	Negative	completed
3	RES-1419610	SP-2867825	Immunohistochemistry (IHC)	PD-L1 TPS	74.07	%	High Expression	completed
4	RES-5614226	SP-1419610	Liquid Biopsy Marker	CA-125	72.42	U/mL	Elevated	completed
5	RES-5108603	SP-1419610	Liquid Biopsy Marker	ctDNA Detected			Negative (No Disease Detected)	completed
6	RES-4744854	SP-5614226	Liquid Biopsy Marker	CA-125	30	U/mL	Normal	completed
7	RES-3341057	SP-5614226	Liquid Biopsy Marker	ctDNA Detected			Positive (Disease Detected)	completed
8	RES-2719583	SP-5108603	Liquid Biopsy Marker	CA-125	141.4	U/mL	Elevated	completed
9	RES-2458591	SP-5108603	Liquid Biopsy Marker	ctDNA Detected			Negative (No Disease Detected)	completed
10	RES-8078673	SP-4744854	NIPT Panel	Fetal Fraction	10.07	%	Sufficient for Analysis	completed

Following is the code to create 'Treatments' table.

```
# Initialize Faker and random seeds

fake = Faker()

Faker.seed(42)

np.random.seed(42)

# 1. Load the upstream data to build precision medicine logic
try:

    df_patients = pd.read_csv('patients_data.csv')

    df_visits = pd.read_csv('clinical_visits_data.csv')

    df_results = pd.read_csv('lab_results_data.csv')

except FileNotFoundError:

    print("Error: Missing CSV files. Please ensure all previous CSVs are in the directory.")

    exit()
```

```
# We need to know which lab results belong to which patient, so we merge via the
specimen table

# First, load specimens

try:

    df_specimens = pd.read_csv('specimens_data.csv')

except FileNotFoundError:

    print("Error: Missing specimens_data.csv.")

    exit()


# Merge: Results -> Specimens -> Visits (gives us patient_id and visit_date)

df_res_spec = df_results.merge(df_specimens[['specimen_id', 'visit_id']], on='specimen_id',
how='left')

df_full = df_res_spec.merge(df_visits[['visit_id', 'patient_id', 'visit_date']], on='visit_id',
how='left')


# Also merge patients to get the cohort

df_master = df_full.merge(df_patients[['patient_id', 'biological_cohort']], on='patient_id',
how='left')


# Convert visit_date to datetime objects for accurate date math

df_master['visit_date'] = pd.to_datetime(df_master['visit_date'])


treatments_data = []


# Keep track of patients who have already been treated so we don't duplicate primary care

treated_patients = set()
```

```

# 2. Iterate through the lab results to prescribe treatments based on the findings
for index, row in df_master.iterrows():

    patient_id = row['patient_id']
    cohort = row['biological_cohort']
    analyte = row['analyte_name']
    interp = row['clinical_interpretation']
    visit_id = row['visit_id']
    visit_date = row['visit_date']

    # Skip if we already assigned a primary treatment plan for this patient (for simplicity)
    if patient_id in treated_patients:
        continue

#The set(): A Python set is a highly efficient way to store unique items.

#The Business Logic: In a real database, a patient might have dozens of lab results from a
#single visit (e.g., a comprehensive tumor panel). We don't want the script to accidentally
trigger 5 different primary chemotherapy regimens

#for the same patient on the same day.

#By adding the patient_id to the treated_patients set the moment a major therapy is
prescribed, the continue statement ensures the script skips

#them if it encounters another actionable lab result later in the loop.

    treatment_id = f"TX-{fake.unique.random_number(digits=6, fix_len=True)}"

    t_cat = ""

    t_name = ""

    outcome = np.nan

```

```
# Prescribe treatment start date 7 to 14 days after the lab visit
start_date = visit_date + timedelta(days=np.random.randint(7, 14))
```

```
# 50% chance the treatment is ongoing (NULL end date)
```

```
is_ongoing = np.random.choice([True, False])
```

```
if is_ongoing:
```

```
    end_date = np.nan
```

```
else:
```

```
    # Treatment lasted between 90 and 180 days
```

```
    end_date = start_date + timedelta(days=np.random.randint(90, 180))
```

#Time-to-Treatment: You can't prescribe a drug before the lab result comes back! The script uses timedelta to add a random window of 7 to 14 days to the visit_date to represent the time it takes for the doctor to review the results and initiate the start_date.

#Handling Current Patients: Not all treatments are finished. The script flips a coin (is_ongoing). If true, the end_date is left as np.nan

#(which becomes NULL in SQL), perfectly representing a patient currently actively receiving the drug. If false, it calculates a realistic duration

#(90 to 180 days) for the treatment to conclude

```
# =====
```

```
# ONCOLOGY PRECISION MEDICINE LOGIC
```

```
# =====
```

```
if cohort == 'oncology cohort':
```

```
    if analyte == 'HER2 Expression' and interp == 'Positive':
```

```

t_cat = 'Targeted Therapy'

t_name = 'Trastuzumab (Herceptin)'

outcome = np.random.choice(['Complete Response', 'Partial Response', 'Stable
Disease'], p=[0.2, 0.6, 0.2]) if not is_ongoing else np.nan

treated_patients.add(patient_id)


elif analyte == 'PD-L1 TPS' and interp == 'High Expression':

    t_cat = 'Immunotherapy'

    t_name = 'Pembrolizumab (Keytruda)'

    outcome = np.random.choice(['Complete Response', 'Partial Response', 'Adverse
Event - Discontinued'], p=[0.3, 0.5, 0.2]) if not is_ongoing else np.nan

    treated_patients.add(patient_id)


elif analyte == 'CA-125' and interp == 'Elevated':

    t_cat = 'Chemotherapy'

    t_name = 'Carboplatin + Paclitaxel'

    outcome = np.random.choice(['Partial Response', 'Disease Progression', 'Stable
Disease'], p=[0.4, 0.4, 0.2]) if not is_ongoing else np.nan

    treated_patients.add(patient_id)

#Targeted Action: The script specifically looks for a Positive result on the HER2 Expression
test. If it finds it,

#it triggers the industry-standard targeted therapy: Trastuzumab.

#If it finds High PD-L1, it triggers Pembrolizumab.


#Clinical Outcomes: It assigns a realistic outcome using standard RECIST criteria
terminology

#("Complete Response", "Partial Response", "Stable Disease", or "Disease Progression").

```

#Notice how it forces the outcome to np.nan if the treatment is_ongoing—because you can't have a final outcome if the treatment isn't finished yet!

```
# =====
```

```
# MATERNAL-FETAL CLINICAL LOGIC
```

```
# =====
```

```
elif cohort == 'maternal_fetal cohort':
```

```
    if analyte in ['Trisomy 21', 'Trisomy 18'] and interp == 'High Risk':
```

```
        t_cat = 'Specialist Referral'
```

```
        t_name = 'Maternal-Fetal Medicine (MFM) High-Risk Monitoring'
```

```
        outcome = 'Routine Care Completed' if not is_ongoing else np.nan
```

```
        treated_patients.add(patient_id)
```

```
elif analyte == 'Protein' and interp in ['Trace', '1+']:
```

```
    t_cat = 'Monitoring / Intervention'
```

```
    t_name = 'Preeclampsia Monitoring Protocol'
```

```
    outcome = 'Routine Care Completed' if not is_ongoing else np.nan
```

```
    treated_patients.add(patient_id)
```

```
elif analyte == 'Fetal Fraction' and interp == 'Sufficient for Analysis':
```

```
    t_cat = 'Prenatal Care'
```

```
    t_name = 'Standard Prenatal Vitamins + Folic Acid'
```

```
    outcome = 'Routine Care Completed' if not is_ongoing else np.nan
```

```
    treated_patients.add(patient_id)
```

The logic shifts completely for the maternal cohort. Instead of prescribing cancer drugs,

#a "High Risk" NIPT result triggers a clinical workflow action: a Specialist Referral to a High-Risk MFM doctor.

#Routine, healthy results (like a sufficient Fetal Fraction with no high-risk markers) trigger standard preventative care: Prenatal Vitamins.

```
# Append if a treatment was successfully triggered
```

```
if t_cat != "":
```

```
    # Convert dates back to strings for CSV export
```

```
    start_date_str = start_date.strftime('%Y-%m-%d')
```

```
    end_date_str = end_date.strftime('%Y-%m-%d') if pd.notna(end_date) else np.nan
```

```
treatments_data.append([
```

```
    treatment_id, patient_id, visit_id, t_cat, t_name,
```

```
    start_date_str, end_date_str, outcome
```

```
])
```

#Finally, before adding the row to the dataset, the script converts the Python datetime objects back into

#standard string formats (%Y-%m-%d) using .strftime().

#This guarantees the dates are formatted exactly the way SQL expects them to be (YYYY-MM-DD) when you import the CSV.

3. Create DataFrame and Export

```
columns = [
```

```
    'treatment_id', 'patient_id', 'visit_id', 'treatment_category',
```

```
    'treatment_name', 'start_date', 'end_date', 'treatment_outcome'
```

```
]
```

```
df_treatments = pd.DataFrame(treatments_data, columns=columns)
```



```
print(df_treatments.head(15))

# Export to CSV, keeping NaNs as empty strings

df_treatments.to_csv('treatments_data.csv', index=False, na_rep='')

print(f"\nSuccessfully generated {len(df_treatments)} treatment records and saved to
'treatments_data.csv'")
```

Sample 'Treatments' table.

	A	B	C	D	E	F	G	H
1	treatment_id	patient_id	visit_id	treatment_category	treatment_name	start_date	end_date	treatment_outcome
2	TX-216739	PT-00001	V-216739	Immunotherapy	Pembrolizumab (Keytruda)	9/12/2024	2/9/2025	Partial Response
3	TX-126225	PT-00002	V-334053	Prenatal Care	Standard Prenatal Vitamins + Folic Acid	11/10/2023		
4	TX-877572	PT-00003	V-876646	Prenatal Care	Standard Prenatal Vitamins + Folic Acid	10/15/2024		
5	TX-388389	PT-00004	V-542417	Targeted Therapy	Trastuzumab (Herceptin)	8/5/2024	11/26/2024	Partial Response
6	TX-334053	PT-00005	V-343962	Immunotherapy	Pembrolizumab (Keytruda)	10/22/2023	2/26/2024	Complete Response
7	TX-207473	PT-00006	V-781453	Chemotherapy	Carboplatin + Paclitaxel	11/9/2023		
8	TX-671858	PT-00007	V-539898	Chemotherapy	Carboplatin + Paclitaxel	1/11/2025		
9	TX-191161	PT-00008	V-948749	Prenatal Care	Standard Prenatal Vitamins + Folic Acid	12/6/2023	5/3/2024	Routine Care Completed
10	TX-719176	PT-00009	V-944962	Targeted Therapy	Trastuzumab (Herceptin)	12/18/2024	5/18/2025	Partial Response

3. Creating the database from these tables in Sqlite.

As we can see, the empty tables with attributes are ready in SQLite and the generated data is there in 5 different excel files. This data is imported into the tables in SQLite so that the empty tables get populated.

This way all the tables are ready with data and with connections among them.

4. Querying the database using SQL

To get the desired information from the database, I first wrote down some questions. Then I answered to those questions by querying the database using SQL as follows.

--1. Write a query to retrieve the patient_id, age, and smoking_status of all patients who belong to the 'maternal_fetal cohort'.

```
SELECT patients.patient_id, patients.age, patients.smoking_status  
FROM patients  
WHERE patients.biological_cohort = "maternal fetal cohort";
```

--We need to audit our active prescriptions to see who is currently receiving care.

-- 2. Write a query to find all active treatments. Return the patient_id, treatment_name, and start_date.

```
SELECT treatments.patient_id, treatments.treatment_name, treatments.start_date  
FROM treatments  
WHERE treatments.end_date = "";
```

-- Scenario: The Lab Manager wants to know how many samples the lab is wasting due to quality issues.

-- 3. Write a query to count how many specimens fall into each qc_status category.

-- Your output should have two columns: the qc_status and the total count of specimens for that status.

```
SELECT specimens.qc_status,  
COUNT (specimen_id) AS total_specimens  
FROM specimens  
GROUP BY specimens.qc_status  
ORDER BY total_specimens DESC;
```

-- Scenario: The research team wants to look at the records of our oldest oncology patients.

-- 4: Write a query to list the patient_id, age, and gender of the 5 oldest patients in the 'oncology cohort'.

```
SELECT patients.patient_id, patients.age, patients.gender  
FROM patients  
WHERE patients.biological_cohort = "oncology cohort"  
ORDER BY patients.age DESC  
LIMIT 5;
```

-- Scenario: We need to map a timeline of when patients had their very first visit.

--5. Write a query that links the Patients table to the Clinical_Visits table.

-- Return the patient_id, biological_cohort, and visit_date specifically for visits where

-- the clinical_information is exactly 'Baseline Initial Consult'.

```
SELECT p.patient_id, p.biological_cohort, v.visit_date
FROM patients as p
INNER JOIN clinical_visits as v
ON p.patient_id = v.patient_id
WHERE v.clinical_information = "Baseline Initial Consult";
```

-- The Lab Director wants to investigate wasted resources specifically within the prenatal program.

-- 6. Write a query to find the patient_id, visit_date, specimen_type, and qc_status of all specimens that have a qc_status of 'fail' or 'rejected', but only for patients in the 'maternal_fetal cohort'.

```
SELECT p.patient_id, v.visit_date, s.specimen_type, s.qc_status
FROM patients AS p
JOIN clinical_visits AS v ON p.patient_id = v.patient_id
JOIN specimens AS s ON s.visit_id = v.visit_id
WHERE s.qc_status IN ('rejected', 'fail')
AND p.biological_cohort = 'maternal fetal cohort';
```

-- Scenario: The clinical administration team is auditing "high-utilization" patients to ensure they are getting enough support.

--7. Write a query to find the patient_id and the total count of visits for any patient who has had more than 3 clinical visits.

```
SELECT
    patient_id,
    COUNT(visit_id) AS total_visits
FROM
    Clinical_Visits
GROUP BY
    patient_id
HAVING
    COUNT(visit_id) > 3;
```

--Scenario: The research team wants a demographic breakdown of the entire database, categorized by age groups rather than individual ages.

-- 8. Write a query that categorizes patients into three groups: 'Under 40', '40 to 60', and 'Over 60'. Return the name of the age group and the total count of patients in that group.

```
SELECT CASE
    WHEN age < 40 THEN 'Under 40'
    WHEN age >= 40 AND age <= 60 THEN '40 to 60'
    WHEN age > 60 THEN 'Over 60'
    END AS age_group,
    COUNT (patient_id) AS total_patients
FROM patients
```

GROUP BY

CASE

WHEN age < 40 THEN 'Under 40'

WHEN age >= 40 AND age <= 60 THEN '40 to 60'

WHEN age > 60 THEN 'Over 60'

END

ORDER BY total_patients DESC;

-- Scenario: The executive board wants to track our "Time-to-Treatment" KPI (Key Performance Indicator) to ensure patients are receiving timely care.

-- 9. Write a query to calculate the time difference in days between a patient's 'Baseline Initial Consult' (from the Clinical_Visits table) and the start_date of their therapy (from the Treatments table).

-- Return the patient_id, the visit_date, the start_date, and the calculated difference in a new column called days_to_treatment.

SELECT v.patient_id, v.visit_date, t.start_date, (t.start_date - v.visit_date) AS
days_to_treatment

FROM clinical_visits AS v

JOIN treatments AS t

ON v.patient_id = t.patient_id

WHERE v.clinical_information = 'Baseline Initial Consult';

-- Scenario: The clinical administration team is auditing active patient rosters and needs a report showing the date of every patient's most recent interaction with the clinic.

-- 10. Write a query that returns the patient_id, biological_cohort, and their most recent visit_date.

```
WITH RankedVisits AS (  
    SELECT  
        p.patient_id,  
        p.biological_cohort,  
        v.visit_date,  
        ROW_NUMBER() OVER (  
            PARTITION BY p.patient_id  
            ORDER BY v.visit_date DESC  
        ) AS visit_rank  
    FROM  
        Patients p  
    INNER JOIN  
        Clinical_Visits v  
        ON p.patient_id = v.patient_id  
)  
SELECT  
    patient_id,  
    biological_cohort,  
    visit_date  
FROM  
    RankedVisits  
WHERE
```

```
visit_rank = 1;
```

```
-- Scenario: The Lab Director suspects that certain surgical procedures are resulting in lower-quality samples than others.
```

```
-- They need a failure rate calculated for every procedure type.
```

```
-- 11. Write a query that calculates the failure rate for each procedure in the Specimens table.
```

```
-- Your output should include the procedure, the total number of specimens collected via that procedure,
```

```
-- the number of failed/rejected specimens, and a calculated column for the failure rate percentage.
```

```
-- Order the results by the highest failure rate first.
```

```
WITH procedure_view AS (
```

```
  SELECT procurement_procedure,
```

```
         COUNT(specimen_id) AS total_specimens,
```

```
         SUM(CASE WHEN qc_status IN ('fail', 'rejected') THEN 1 ELSE 0 END) AS  
failed_specimens
```

```
        FROM specimens
```

```
        WHERE specimen_id IS NOT NULL
```

```
        GROUP BY procurement_procedure)
```

```
SELECT procurement_procedure,
```

```
       total_specimens,
```

```
       failed_specimens, ROUND((failed_specimens * 100) / total_specimens, 2) AS  
failure_rate_percent
```

```
FROM procedure_view
```


ORDER BY failure_rate_percent DESC;

-- Scenario: The care coordination team needs to find patients who came in for their initial consultation but slipped through the cracks and never started a therapy.

--12. Write a query to retrieve the patient_id, age, and biological_cohort of all patients who had a visit with the clinical_information of 'Baseline Initial Consult', but who do not have any records existing in the Treatments table.

```
SELECT p.patient_id, p.age, p.biological_cohort
FROM patients AS p
INNER JOIN clinical_visits as c
ON p.patient_id = c.patient_id
WHERE c.clinical_information = 'Baseline Initial Consult'
AND NOT EXISTS (SELECT 1 FROM treatments t
                 WHERE p.patient_id = t.patient_id);
```

-- Scenario: The oncology research team is tired of reading the Lab_Results table row-by-row. They want a "flattened" patient-level view where each major biomarker has its own dedicated column.

-- 13. Write a query that returns the patient_id alongside two brand new columns: her2_status and pdl1_status.

-- These columns should contain the clinical_interpretation for the 'HER2 Expression' and 'PD-L1 TPS' analytes, respectively.

```
SELECT p.patient_id,
MAX(CASE WHEN r.analyte_name = "HER2 Expression" THEN r.clinical_interpretation END)
AS her2_status,
MAX(CASE WHEN r.analyte_name = "PD-L1 TPS" THEN r.clinical_interpretation END) AS
pdl1_status
```

```
FROM patients p
INNER JOIN clinical_visits v ON p.patient_id = v.patient_id
INNER JOIN specimens s ON s.visit_id = v.visit_id
INNER JOIN lab_results r ON r.specimen_id = s.specimen_id
WHERE r.analyte_name IN ("HER2 Expression", "PD-L1 TPS")
GROUP BY p.patient_id;
```